

PROJECT ASSIGNMENT

GeminiDoc-Insights with Vector Database Integration

Student ID: 2BerbyMarty2

Course: Advanced AI/ML Development

Academic Year: 2025–2026 — Submission Deadline: May 31, 2026

Executive Summary

This assignment requires the enhancement and completion of the **GeminiDoc-Insights** project by integrating a vector database to enable semantic search, retrieval-augmented generation (RAG), and intelligent document analysis capabilities. The project demonstrates proficiency in cloud APIs, machine learning embeddings, database design, and full-stack development.

Contents

1	Project Overview	3
1.1	Current State	3
1.2	Project Enhancement Objective	3
2	Learning Objectives	3
3	Assignment Tasks	3
3.1	Phase 1: Vector Database Infrastructure (40 points)	3
3.1.1	Task 1.1: Select and Initialize Vector DB (10 points)	3
3.1.2	Task 1.2: Implement Embedding Generation (15 points)	4
3.1.3	Task 1.3: Store in Vector DB (15 points)	5
3.2	Phase 2: RAG Pipeline Implementation (35 points)	5
3.2.1	Task 2.1: Semantic Search (10 points)	6
3.2.2	Task 2.2: Context-Aware Generation (15 points)	6
3.2.3	Task 2.3: Conversation Memory (10 points)	7
3.3	Phase 3: Advanced Features & Documentation (15 points)	7
3.3.1	Task 3.1: Enhanced Search Capabilities (5 points)	7

3.3.2	Task 3.2: Comprehensive Documentation (5 points)	8
3.3.3	Task 3.3: Testing and Validation (5 points)	8
4	Technical Specifications	8
4.1	Technology Stack	9
4.2	System Architecture	9
4.3	Configuration Requirements	9
5	Evaluation Criteria	10
5.1	Grading Rubric (Total: 90 points)	10
5.2	Additional Scoring Factors	10
6	Submission Requirements	10
6.1	Repository Structure	10
6.2	Submission Checklist	11
6.3	Git Commit Requirements	11
7	Resources and References	11
7.1	Required APIs and Services	11
7.2	Recommended Learning Materials	12
8	Important Notes	12
9	Frequently Asked Questions	12
9.1	Q: Which vector database should I choose?	12
9.2	Q: Can I use multiple data sources simultaneously?	12
9.3	Q: What if I exceed API rate limits?	12
9.4	Q: How do I handle long documents?	12
10	Contact and Support	13

1 Project Overview

1.1 Current State

The GeminiDoc-Insights repository (2BerbyMarty2/GeminiDoc-Insights) currently contains:

- A Google Colab notebook with partial implementation
- YouTube transcript extraction functionality
- PDF document processing via Gemini File API
- Text chunking using Langchain's `RecursiveCharacterTextSplitter`
- An incomplete final implementation cell

1.2 Project Enhancement Objective

Extend the existing notebook to include a **production-grade vector database system** that enables:

1. Semantic search across document collections
2. Retrieval-Augmented Generation (RAG) for intelligent Q&A
3. Persistent storage of embeddings and metadata
4. Multi-document similarity analysis
5. Interactive query interface

2 Learning Objectives

Upon completion of this assignment, you will demonstrate:

- **Vector Embeddings:** Understanding of semantic embeddings and their generation
- **Database Design:** Schema design for vector and metadata storage
- **RAG Pipelines:** Implementation of retrieval-augmented generation systems
- **API Integration:** Working with multiple APIs (Gemini, vector DB services)
- **Software Engineering:** Code organization, documentation, and testing
- **Production Deployment:** Creating reproducible, scalable solutions

3 Assignment Tasks

3.1 Phase 1: Vector Database Infrastructure (40 points)

3.1.1 Task 1.1: Select and Initialize Vector DB (10 points)

Objective: Evaluate and implement a vector database solution.

Requirements:

1. Compare at least 3 vector database options:
 - Pinecone (cloud-based, managed service)
 - Weaviate (open-source, flexible)
 - Chroma (lightweight, embedded)
 - Milvus (high-performance)
 - FAISS (Facebook's local library)
2. Create a comparison document (Markdown) with:
 - Pros and cons of each option
 - Cost analysis
 - Scalability considerations
 - Recommended choice with justification
3. Implement initialization code for chosen database
4. Document API authentication and connection setup

Deliverables:

- VECTOR_DB_COMPARISON.md
- Database initialization code in notebook
- Documentation of setup process

3.1.2 Task 1.2: Implement Embedding Generation (15 points)

Objective: Create embeddings from chunked text content.

Requirements:

1. Design embedding pipeline that:
 - Takes chunked text as input
 - Generates vector embeddings (384-1536 dimensions)
 - Preserves metadata (source, chunk index, timestamp)
 - Handles batch processing for efficiency
2. Select embedding model (justify choice):
 - Google's Embedding API (via Gemini)
 - Sentence-transformers (open-source)
 - OpenAI embeddings
 - Alternative models
3. Implement error handling for:
 - Rate limiting

- Token overflow
- API failures
- Timeout scenarios

4. Add performance monitoring (embedding time, throughput)

Deliverables:

- Embedding generation function
- Batch processing capability
- Error handling mechanism
- Performance metrics logging

3.1.3 Task 1.3: Store in Vector DB (15 points)

Objective: Persist embeddings and metadata to vector database.

Requirements:

1. Design database schema including:
 - Vector field (embeddings)
 - Metadata fields (document ID, chunk index, source type, timestamp)
 - Text content storage
 - Collection/index structure
2. Implement storage functions:
 - Upload embeddings and metadata
 - Create collections per document
 - Handle duplicate prevention
3. Add query indexing for performance
4. Implement data persistence verification
5. Create backup/export functionality

Deliverables:

- Database schema documentation
- Storage and retrieval functions
- Data verification tests
- Export/backup utilities

3.2 Phase 2: RAG Pipeline Implementation (35 points)

3.2.1 Task 2.1: Semantic Search (10 points)

Objective: Implement similarity-based document retrieval.

Requirements:

1. Create query embedding function
2. Implement similarity search with configurable parameters:
 - Top-K retrieval (default K=5)
 - Similarity threshold
 - Metadata filtering
3. Add ranking and scoring mechanisms
4. Include result annotation (relevance scores)
5. Handle edge cases (empty results, all documents, etc.)

Deliverables:

- Similarity search function
- Configurable parameters
- Result formatting and scoring

3.2.2 Task 2.2: Context-Aware Generation (15 points)

Objective: Implement RAG workflow using retrieved context.

Requirements:

1. Design RAG pipeline:
 - Query embedding creation
 - Retrieve top-K relevant chunks
 - Construct context prompt
 - Send to Gemini API with context
 - Generate answer with source citations
2. Implement context window management:
 - Token counting for prompt + context
 - Context truncation if needed
 - Priority-based chunk selection
3. Add citation tracking:
 - Source document identification
 - Chunk reference information
 - Confidence scoring

4. Support multiple query types:

- Direct Q&A
- Summarization
- Comparative analysis
- Keyword extraction

Deliverables:

- RAG pipeline implementation
- Context management system
- Citation and reference system
- Multi-query type support

3.2.3 Task 2.3: Conversation Memory (10 points)

Objective: Support multi-turn conversations with memory.

Requirements:

1. Implement conversation history tracking
2. Store previous queries and answers
3. Use history in subsequent queries (context awareness)
4. Implement conversation reset functionality
5. Add conversation export/logging
6. Limit history size to manage token usage

Deliverables:

- Conversation memory system
- History tracking mechanism
- Conversation export functionality

3.3 Phase 3: Advanced Features & Documentation (15 points)

3.3.1 Task 3.1: Enhanced Search Capabilities (5 points)

Requirements:

1. Implement metadata filtering (document type, date range, source)
2. Add hybrid search (keyword + semantic combined)
3. Support advanced filtering queries

3.3.2 Task 3.2: Comprehensive Documentation (5 points)

Requirements:

1. Expand README with:
 - Project overview and capabilities
 - Setup instructions (step-by-step)
 - API key configuration
 - Usage examples with expected outputs
 - Troubleshooting guide
 - Architecture diagram
2. Add inline code documentation (docstrings)
3. Create architecture documentation

Deliverables:

- Enhanced README.md
- ARCHITECTURE.md (system design)
- Function docstrings and comments

3.3.3 Task 3.3: Testing and Validation (5 points)

Requirements:

1. Create test cases for:
 - Embedding generation
 - Database storage and retrieval
 - Similarity search accuracy
 - RAG pipeline end-to-end
 - Error scenarios
2. Document test results
3. Include usage examples with real data

Deliverables:

- Test notebook or test script
- Test results and coverage report
- Example usage demonstrations

4 Technical Specifications

Component	Technology	Purpose
Text Chunking	Langchain	Document preprocessing
Embeddings	Google Gemini API or Sentence-Transformers	Vector generation
Vector DB	Pinecone/Weaviate/Chroma/FAISS	Semantic search storage
LLM	Google Gemini	Answer generation
Environment	Google Colab	Development and execution

4.1 Technology Stack

4.2 System Architecture

```
# Data Flow
User Input (YouTube/PDF)
  ->
Text Extraction & Chunking (Langchain)
  ->
Embedding Generation (Gemini/Transformers)
  ->
Vector DB Storage (Pinecone/Weaviate/Chroma)
  ->
Query Processing
  +- Query Embedding
  +- Semantic Search
  +- Context Retrieval
  '- RAG with Gemini
  ->
Answer Generation with Citations
```

Listing 1: High-Level Architecture

4.3 Configuration Requirements

Environment Variables:

```
GEMINI_API_KEY=your_api_key_here
VECTOR_DB_API_KEY=your_db_api_key
VECTOR_DB_INDEX_NAME=geminidoc_insights
EMBEDDING_MODEL=google # or sentence-transformers
CHUNK_SIZE=1000
CHUNK_OVERLAP=100
TOP_K_RETRIEVAL=5
MAX_CONTEXT_TOKENS=3000
```

5 Evaluation Criteria

5.1 Grading Rubric (Total: 90 points)

Criterion	Points	Description
Vector DB Implementation	40	Proper setup, initialization, embedding generation, and storage
RAG Pipeline	35	Similarity search, context retrieval, answer generation
Documentation & Testing	15	Code clarity, documentation completeness, test coverage

5.2 Additional Scoring Factors

- **Code Quality** (+5 bonus): Clean, modular, well-commented code
- **Performance Optimization** (+5 bonus): Efficient batch processing, caching strategies
- **UI/Frontend** (+5 bonus): Streamlit or interactive interface
- **Advanced Features** (+5 bonus): Unique enhancements beyond requirements

6 Submission Requirements

6.1 Repository Structure

```
2BerbyMarty2/GeminiDoc-Insights/
+-- Gemini_API.ipynb           # Updated main notebook
+-- README.md                 # Enhanced documentation
+-- ARCHITECTURE.md           # System design documentation
+-- VECTOR_DB_COMPARISON.md   # Vector DB analysis
+-- tests/
|   +-- test_embeddings.py     # Embedding tests
|   +-- test_vector_db.py     # Database tests
|   '-- test_rag_pipeline.py  # RAG pipeline tests
+-- config/
|   '-- config.py             # Configuration management
+-- src/
|   +-- embedding_engine.py    # Embedding generation
|   +-- vector_store.py       # Vector DB interface
|   +-- rag_pipeline.py       # RAG implementation
|   '-- utils.py              # Helper functions
+-- examples/
|   '-- usage_examples.ipynb  # Example usage
'-- ASSIGNMENT.md             # This file (converted to
    Markdown)
```

6.2 Submission Checklist

Before submitting, verify:

- ✓ All Phase 1 tasks completed (40 points)
- ✓ All Phase 2 tasks completed (35 points)
- ✓ All Phase 3 tasks completed (15 points)
- ✓ README updated with setup and usage instructions
- ✓ Code is well-commented and documented
- ✓ All tests pass successfully
- ✓ Example notebooks demonstrate functionality
- ✓ Repository is organized and clean
- ✓ All API keys properly configured (not exposed in code)
- ✓ Commit history shows progressive development

6.3 Git Commit Requirements

Make meaningful commits with clear messages:

```
git commit -m "Phase 1.1: Initialize vector DB infrastructure"
git commit -m "Phase 1.2: Implement embedding generation pipeline"
git commit -m "Phase 1.3: Add vector database storage functions"
git commit -m "Phase 2.1: Implement semantic search"
git commit -m "Phase 2.2: Build RAG pipeline with context retrieval"
git commit -m "Phase 2.3: Add conversation memory system"
git commit -m "Phase 3: Documentation and testing"
```

7 Resources and References

7.1 Required APIs and Services

- Google Gemini API: <https://ai.google.dev>
- YouTube Transcript API: <https://github.com/jdepoix/youtube-transcript-api>
- Langchain Documentation: <https://python.langchain.com>
- Vector DB Options:
 - Pinecone: <https://www.pinecone.io>
 - Weaviate: <https://weaviate.io>
 - Chroma: <https://www.trychroma.com>
 - Milvus: <https://milvus.io>
 - FAISS: <https://github.com/facebookresearch/faiss>

7.2 Recommended Learning Materials

1. RAG System Design: *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*
2. Vector Embeddings: *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*
3. Langchain Guide: <https://python.langchain.com/docs>
4. Gemini API Tutorials: <https://ai.google.dev/tutorials>

8 Important Notes

- **API Costs:** Be mindful of API usage costs. Use free tiers when available.
- **Rate Limiting:** Implement proper rate limiting and backoff strategies.
- **Data Privacy:** Do not commit API keys or sensitive data to the repository.
- **Testing:** Thoroughly test all components before final submission.
- **Scalability:** Design for scalability—your solution should handle 1000+ document chunks.

9 Frequently Asked Questions

9.1 Q: Which vector database should I choose?

A: For simplicity and quick prototyping, use **Chroma** (works locally). For production and scalability, use **Pinecone** or **Weaviate**. FAISS is excellent for local, high-performance needs.

9.2 Q: Can I use multiple data sources simultaneously?

A: Yes! Your system should handle multiple YouTube videos and PDF documents. Implement collection-based organization.

9.3 Q: What if I exceed API rate limits?

A: Implement exponential backoff, caching, and request queuing. Document this in your solution.

9.4 Q: How do I handle long documents?

A: Adjust chunk size and overlap parameters. Use the token counting feature to ensure chunks fit within model limits.

10 Contact and Support

For questions or clarifications:

- Review the assignment in detail
- Check the resources section
- Consult official API documentation
- Create GitHub issues for technical problems

Due Date: As specified by your instructor

Late Submission: -5 points per day

Academic Integrity: Cite all external resources and collaborations